RESULTADO EN PROYECTO QGIS



CAPTURAS DEL CÓDIGO EN SOFTWARE R

U3_script_bosque_implantado_2025.R

Source on Save

```r
# Curso: Herramientas avanzadas para la teledetección de recursos agrícolas y forestales #
# Septiembre 2025                                                                         #
# Instituo Gulich                                                                         #
# Eduardo Hildt (hildt.eduardo@inta.gob.ar)                                               #
###########################################################################################
# Unidad 3: Analisis de un rodal de pino implantado.

# Primera parte basada en:
# Aplicações do lidar para o inventário florestal - enfoque
# unidade de área / Ana Paula Dalla Corte, Eben North
# Broadbent, Lucas Henderson de Oliveira Santos, et al. -
# Ponta Grossa - PR, 2022.
# Modificado para emplear el enfoque de árbol individual

#############################################################################
# Dependencias requeridas para instalar paquetes
# Ubuntu/Debian
#       libudunits2-dev
#       libgdal-dev
#       libgl1-mesa-dev
#       libglu1-mesa-dev
#       libgeos-dev
#       libproj-dev
#       r-cran-rgl (sudo apt-get build-dep r-cran-rgl)
#       sudo apt install build-essential r-cran-raster r-cran-terra
#############################################################################

# Instalación de paquetes
#install.packages("lidR")
#install.packages("rgl")
#install.packages("ggplot2")
#install.packages("sf")
#install.packages("sp")
#install.packages("terra")
```

```r
# 6 - Generación de un Modelo Digital de Elevación (DEM)
DEM <- rasterize_terrain(datos_clasf, res = 1, algorithm = tin())
plot(DEM, main = "DEM (m)", col = height.colors(50))

# Suavizar el DEM.
# Primero eligiendo el punto mínimo en un área de 9x9 pixeles y luego con un filtro de 15x15 pixeles.
smooth_DEM <- focal(DEM, w = matrix(1, 9, 9), min)
smooth_DEM <- focal(smooth_DEM, w = matrix(1, 15, 15), mean)
plot(smooth_DEM, main = "DEM suavizado (m)", col = height.colors(100))

# Generación de curvas de nivel
# Definir rango de elevaciones y equidistancia
elevacion_min <- 0
elevacion_max <- 200
equidistancia <- 0.5

# Calcular niveles de elevación
niveles_elevacion <- seq(elevacion_min, elevacion_max, by = equidistancia)
curvas_nivel <- as.contour(smooth_DEM, levels = niveles_elevacion)

# Visualizar DEM con curvas de nivel superpuestas
plot(smooth_DEM, main = "DEM con Curvas de Nivel", col = height.colors(50))
contour(smooth_DEM, levels = niveles_elevacion, add = TRUE)

#############################################################################
```

```r
# 12 - Segmentación de la nube de puntos
# Segmentar nube de puntos para identificar árboles individuales ####
# Configurar algoritmo de segmentación Dalponte2016
algoritmo_segmentacion <- dalponte2016(
  chm = CHM,                          # Modelo de altura de copas
  treetops = arboles_3m,                 # Ubicación de árboles detectados
  th_tree = 6,                        # Umbral mínimo de altura de árbol
  th_seed = 0.45,                     # Umbral para semillas
  th_cr = 0.55,                       # Umbral para crecimiento de copa
  max_cr = 6,                         # Máximo radio de copa (píxeles)
  ID = "treeID"                       # Campo ID para identificación
)

datos_segmentados <- segment_trees(datos_norm, algoritmo_segmentacion)

# Visualizar árboles segmentados (OJO: Tarda mucho tiempo en presentar el gráfico)
plot(datos_segmentados, bg = "white", size = 4,
                        color = "treeID", pal = random.colors,
                        backend = "rgl", axis = T, legend = F)

################################################################################

# 13 - Visualización de árboles individuales
# Graficar un árbol individual
id_buscado <- 100
arbol_i <- filter_poi(datos_segmentados, treeID == id_buscado)
plot(arbol_i, color = "Z", size = 8, bg = "white", axis = TRUE, legend = TRUE)
```

```r
# Importar el archivo vectorial que contiene los rodales (polígonos)
# Incluyendo su área y el valor medio del DAP
rodales <- vect("./rodales_pino.shp")

# Graficar el CHM y los poligonos
plot(CHM)
plot(rodales, add = TRUE, border =  "darkmagenta", lwd = 2)

# Importar el archivo que contiene la localización de los árboles.
arboles_3m <- vect("./Salidas_VECTORIALES/Arboles.shp")
arboles_3m <- as(arboles_3m, "Spatial")

#####################################################################

# Definir un raster que se empleará para resumir los datos
resolucion = 25
rst <- raster::raster(crs = crs(CHM), res = resolucion)
rst <- raster::setExtent(rst, arboles_3m@bbox, keepres = TRUE)
```

```r
# 12. Imprimir resultados en consola
print("RESULTADOS DEL ANÁLISIS AGROSILVOPASTORIL")
print("========================================")
print(paste("Área del ROI:", round(area_roi_ha, 2), "ha"))
print(paste("Número total de árboles:", num_arboles))
print(paste("Densidad de árboles:", round(densidad_arboles, 2), "árboles/ha"))
print(paste("Media de densidad por celda:", round(media_densidad, 2), "árboles/ha"))
print(paste("Varianza de densidad por celda:", round(varianza_densidad, 2)))
print(paste("Área total de copas:", round(area_total_copas_ha, 4), "ha"))
print(paste("Porcentaje de cobertura de copas:", round(porcentaje_cobertura, 2), "%"))
print(paste("Área media de copa por árbol:", round(area_media_copa, 2), "m²"))
print(paste("Varianza del área de copa:", round(varianza_area_copa, 2), "m⁴"))

# 13. Generar gráficos adicionales (opcional)
# Histograma de áreas de copa
png("histograma_areas_copa.png", width = 800, height = 600)
hist(areas_copa_m2, main = "Distribución de áreas de copa",
     xlab = "Área de copa (m²)", ylab = "Frecuencia", col = "lightgreen")
dev.off()

# Mapa de densidad de árboles
png("mapa_densidad_arboles.png", width = 800, height = 600)
plot(st_geometry(roi), main = "Densidad de árboles por celda")
plot(grid, add = TRUE, border = "gray")
colores <- colorRampPalette(c("blue", "green", "yellow", "red"))(10)
plot(grid, add = TRUE,
     col = colores[cut(densidad_por_celda, breaks = 10, labels = FALSE)],
     border = NA)
legend("topright", legend = paste0(round(seq(min(densidad_por_celda),
                                             max(densidad_por_celda),
                                             length.out = 5), 1),
                                   " árboles/ha"),
       fill = colores[c(1, 3, 5, 7, 9)])
dev.off()
```